Student Learning Activity and Course Relationship Tracker

Graduate NoSQL Database Project Report

Neo4j Graph Database • Python Flask Web Application

1. Executive Summary and Introduction

The Student Learning Activity and Course Relationship Tracker is a web-based system that manages students, courses, enrollments, and learning activities using Neo4j, a NoSQL graph database. Its central goal is to connect educational records through explicit relationships rather than treating each record as an isolated row. Users can manage records, enroll students, log activities, and review learning history through a browser-based interface.

The application uses Python and Flask for request handling and database operations; HTML, CSS, and JavaScript provide the user interface. Cypher queries create, retrieve, update, and delete graph data. The project demonstrates why a graph database is a practical fit for interconnected academic information.

2. Problem Statement

A learning-management environment must answer questions that span multiple entities: Which courses is a student enrolled in? Which activities did the student complete? Which course does each activity belong to? Traditional tables can store these facts, but following their connections typically requires multiple joins. This project models those connections directly and provides a small management application to demonstrate them.

3. Project Objectives

- Maintain student and course records with create, view, edit, and delete operations.
- Represent student-to-course enrollment and activity relationships in Neo4j.
- Record learning activities with title, type, date, duration, and description.
- Display student learning history and summary metrics.
- Validate user inputs and protect the consistency of graph relationships.
- Document the installation and recreation process in a public GitHub repository.

4. Scope

The application is designed as a local academic demonstration rather than a production learning-management platform. It does not include institutional login, hosted deployment, or integrations with university systems. Its scope emphasizes graph modeling, Cypher queries, CRUD functionality, and a working end-to-end demonstration.

5. Technologies and Architecture

Layer	Technology	Role
Database	Neo4j Desktop / Neo4j	Stores nodes, properties, and relationships
Database language	Cypher	Graph queries and updates
Backend	Python / Flask	Routes, validation, application logic

Database driver	Neo4j Python driver	Connects Flask to Neo4j via Bolt
Frontend	HTML / CSS / JavaScript	Forms, tables, modals, dynamic dropdowns
Development	VS Code / Git / GitHub	Coding, version control, documentation

5.1 Request and Data flow

A user opens the Flask application in a browser and submits a form or requests a page. Flask reads and validates the request, executes parameterized Cypher through the Neo4j Python driver, and renders an HTML template with the results. JavaScript is used for interactive features such as course selection and confirmation dialogs.

Browser → Flask route → Neo4j driver → Neo4j graph → Flask template → Browser

5.2 Local Environment

The system runs locally on a Mac using Neo4j Desktop and a Python virtual environment. The browser interface is typically available at http://127.0.0.1:5000 while Flask is running. Neo4j connection credentials are supplied to the application through an environment variable rather than committed to source control.

6. Graph Database Design

Node label	Key properties	Purpose
Student	Student_id, name, email	Identifies a learner
Course	Course_id, course_name, credits	Identifies an academic course
learningActivity	Activity_id, title, activity_type, activity_date, duration, description	Stores a completed learning task

Relationship	From -> To	Meaning
ENROLLED_IN	Student -> Course	Student is registered in courses
COMPLETED	Student -> LearningActivity	Student performed activity
FOR_COURSE	LearningActivity -> Courses	Activity belongs to course

The model allows navigation from a student to enrolled courses and from the student through completed activities to the related course. Unique identifier constraints are included in the setup script.

7. Core System Implementation

7.1 Student management

Users can add, list, edit, and delete student records. Input validation rejects missing fields, names containing numbers or symbols, invalid basic email formats, and duplicate student IDs or email addresses. Validation errors appear on the relevant page using Flask flash messages. Deleting a student also removes associated completed learning activities and their graph relationships.

7.2 Course management

Users can add, list, edit, and delete courses. Duplicate course IDs are rejected on creation; credits must be a positive integer. Custom validation feedback is shown within the add and edit forms. The interface uses a confirmation modal for deletion.

7.3 Enrollment management

Enrollment is represented by an ENROLLED_IN relationship between Student and Course nodes. The interface supports adding and removing enrollments. A View Courses action shows the courses associated with a selected student.

7.4 Learning activity management

Each activity stores its identifier, title, type, date, duration in minutes, and description. When an activity is created, the system links it to the selected student through COMPLETED and to a course through FOR_COURSE. The add form dynamically displays only courses in which the selected student is enrolled. The backend also checks enrollment before updating an activity, preventing invalid relationships.

Activity creation checks for an existing activity ID, verifies the selected enrollment relationship, and rejects empty required fields and nonpositive durations. Editing includes required-field, duration, and enrollment validation. The interface includes delete confirmation and an existing popup for failed activity creation

7.5 Dashboard and learning activity

The dashboard displays current counts of students, courses, activities, and relationships. The learning history view summarizes a student's completed activities, total study minutes, and courses studied. These values are derived from the graph rather than entered manually.

8. Example Cypher Operation

Create or find an enrollment

```
MATCH (s:Student {student_id: $student_id}), (c:Course {course_id: $course_id})
MERGE (s)-[:ENROLLED_IN]->(c)
```

Retrieve student activity history

```
MATCH (s:Student {student_id: $student_id})-[:COMPLETED]->(a:LearningActivity)-[:FOR_COURSE]-
>(c:Course)
RETURN s.name, a.title, a.duration, c.course_name
```

9. NoSQL Database Operation

Cypher operation	How it is used
CREATE	Creates new Student, Course, and LearningActivity nodes and activity links
MATCH	Finds existing nodes and traverses relationships
MERGE	Adds enrollment relationships without duplicating an existing edge

SET	Updates student, course, and activity properties
DELETE / DETACH DELETE	Removes relationships and records safely
RETURN / aggregation	Supplies lists, graph counts, and learning history summaries

The database is queried through the official Neo4j Python driver using parameter placeholders such as $student_id. This separates user values from query text and makes the code easier to maintain.

10. Database Integration and Reproducibility

10.1 Installation outline

- Install Neo4j Desktop and create a local Neo4j instance.
- Start the instance and open its query interface.
- Run the Cypher statements from neo4j_setup.cypher in order to create constraints, nodes, and relationships.
- Clone the GitHub repository and create a Python virtual environment.
- Install dependencies using pip install -r requirements.txt.
- Set the NEO4J_PASSWORD environment variable for the local instance.
- Run python app.py and open the local Flask address in a browser.
- Use neo4j_queries.cypher to inspect nodes, relationships, and example graph queries.

10.2 Repository artifacts

The repository includes app.py, HTML templates, requirements.txt, README.md, neo4j_setup.cypher, neo4j_queries.cypher, and supporting project materials. The .gitignore excludes the local virtual environment and environment files. The README provides step-by-step recreation instructions.

10.3 Important verification notes

A fresh-instance recreation test should be performed before final submission to confirm that the setup script and README reproduce the expected graph. The report describes the intended process; it does not claim that an independent fresh-install test has already been completed.

11. Testing and Observed Results

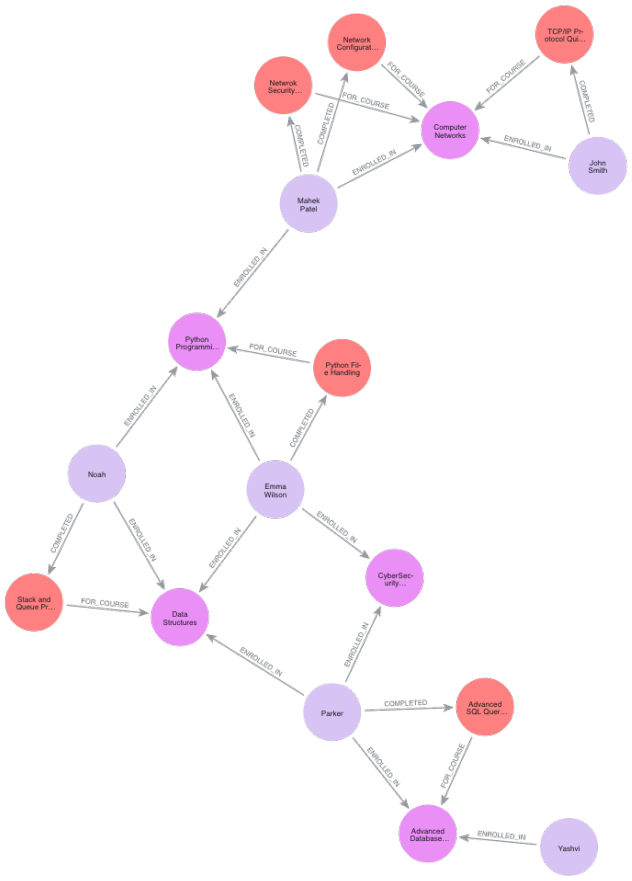
Functional testing was performed through the local web interface while connected to the Neo4j database. Student, course, and activity records were displayed and edited; validation messages were checked in the browser; and updated source code was committed and pushed to GitHub.

Test case	**Expected behavior**	**Status**
Student name contains digits	Reject and show message on Students page	Observed working
Invalid student email	Reject and show message on Students page	Observed working
Duplicate student ID	Reject instead of overwriting record	Observed working
Invalid course credits	Reject and show page-level message	Observed working

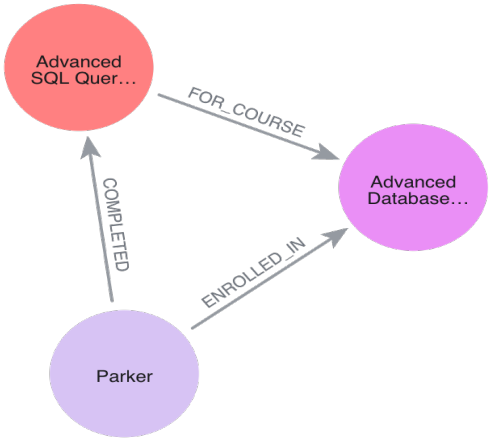
Negative activity duration	Reject and show page-level message	Observed working
Empty activity description on edit	Reject and retain saved data	Observed working
Fresh database recreation	Rebuild sample nodes and relationships	Pending independent verification

11.1 Sample graph summary

Graph 1: Neo4j Graph Showing Student, Course, and Learning Activity Relationships



Graph 2: Student Learning Relationship Graph



Relationships:

- Parker → ENROLLED_IN → Advanced Database Systems
- Parker → COMPLETED → Advanced SQL Query Practice
- Advanced SQL Query Practice → FOR_COURSE → Advanced Database Systems

12. Design Decisions, Limitations, and Future Work

12.1 Why Neo4j?

Neo4j makes connections explicit and easy to traverse. A student can connect to many courses and activities, while each activity points to its course. This structure supports relationship-centered questions without constructing a long chain of relational joins. Cypher also provides readable patterns for explaining and demonstrating the database in a presentation.

12.2 Current limitations

- The application is designed for local use and does not include user accounts or role-based access.
- Email validation is intentionally basic and is not a complete verification system.
- Some HTML inputs use browser validation while others use server-side messages.
- The example database and setup script should be checked together in a clean Neo4j instance before submission.
- No automated test suite or hosted deployment is included in the current scope.

12.3 Protentional improvements

- Introduce user authentication and permissions for students, instructors, and administrators.
- Add automated tests for all routes, including edge cases and invalid relationships.
- Provide learning analytics such as activity trends by course and time period.
- Improve form validation consistency and preserve entered values after errors.
- Deploy the application securely with environment-based configuration.

13. Conclusion

The Student Learning Activity and Course Relationship Tracker demonstrate a complete educational use case for a NoSQL graph database. Neo4j stores interconnected students, courses, enrollments, and activities, while Flask provides a practical interface for maintaining and exploring that data. The project includes graph CRUD operations, learning history, user-input validation, and reproducibility materials on GitHub. It illustrates how a graph model can simplify the representation and retrieval of relationship-rich academic information.

14. Project Repository

GitHub: https://github.com/Mahek1101/Student-Learning-Activity-Tracker

Key files: README.md (setup and recreation), neo4j_setup.cypher (sample graph), neo4j_queries.cypher (demonstration queries), app.py (Flask routes), templates/ (user interface), and requirements.txt (dependencies).

Project Members:

Mahek Patel

Yashvi Patel

Rishikesh Pathak